

Dev Containers

Tu propio entorno de trabajo dentro de un contenedor

Por qué un Dev Container

Instalar Java, Node o cualquier herramienta del curso directamente en tu ordenador tiene los mismos problemas que ya has visto con Docker: una versión distinta a la de tu compañero, un sistema operativo distinto, media mañana perdida configurando un portátil nuevo.

Hasta ahora has usado Docker para meter un servicio dentro de un contenedor, como una base de datos.

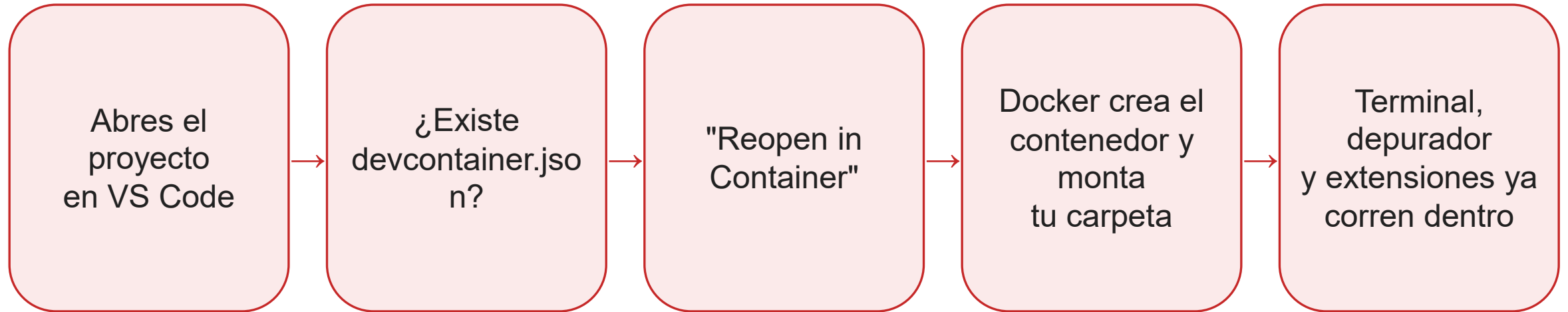
Un Dev Container aplica la misma idea a otra cosa: mete dentro del contenedor tu propio entorno de programación, y tu editor se conecta ahí para trabajar como si ese contenedor fuera tu ordenador.

La parte que cuesta entender la primera vez

“¿Cómo que mi editor está dentro de un contenedor, si lo sigo viendo en mi propia pantalla?” La clave está en separar dos cosas: la ventana que ves (menús, pestañas, el propio VS Code) se queda en tu ordenador, igual que siempre. Pero todo lo que hace falta para trabajar de verdad —terminal, comandos, compilador, extensiones— corre dentro del contenedor.

Como un escritorio remoto: la pantalla, el ratón y el teclado son tuyos, pero el trabajo de verdad ocurre en la máquina remota. Aquí, la "máquina remota" es un contenedor Docker en tu propio ordenador.

Qué ocurre paso a paso



La ventana de VS Code no cambia de aspecto en ningún momento de este proceso: lo único que delata que estás dentro es una etiqueta en la esquina inferior izquierda.

Cómo se nota que estás dentro

Lo único que delata que estás dentro de un contenedor es una etiqueta en la esquina inferior izquierda de VS Code, con el nombre del Dev Container activo.

Lo vas a comprobar tú mismo en la Actividad 0.7: compararás qué responde la terminal dentro del contenedor frente a la terminal de tu propio sistema operativo.

El fichero que lo describe todo

Todo lo anterior lo define un único fichero, `.devcontainer/devcontainer.json`, en la raíz del proyecto:

```
{
  "name": "Entorno del curso",
  "image": "mcr.microsoft.com/devcontainers/base:debian-12",
  "forwardPorts": [5432],
  "postCreateCommand": "echo 'Entorno listo'",
  "customizations": {
    "vscode": {
      "extensions": ["ms-python.python"]
    }
  }
}
```

Qué hace cada campo

image

- la imagen que se usa para construir el contenedor donde vas a trabajar — aquí no aloja una base de datos, sino un sistema base para programar.

forwardPorts

- te deja ver desde tu navegador algo que corre dentro del contenedor.

postCreateCommand

- un comando que se ejecuta una única vez, justo tras crear el contenedor — típico para instalar dependencias.

customizations.vscode.extensions

- instala solas, dentro del contenedor, las extensiones que necesitas.

También puede apoyarse en Docker Compose

En vez de `image`, un `devcontainer.json` puede usar `dockerComposeFile` y `service` para decir "mi entorno de desarrollo es este servicio concreto de este `docker-compose.yml`".

Así, el mismo fichero de Compose que levanta tu base de datos puede levantar, en otro de sus servicios, el contenedor donde vive tu editor: los dos usos de Docker conviven en el mismo fichero.

La relación con Docker, en una frase

Dev Containers no sustituye a Docker: se apoya en él. Docker crea y ejecuta el contenedor; Dev Containers le dice a tu editor cómo conectarse y usarlo como tu ordenador de toda la vida.

	Contenedor de servicio	Dev Container
¿Qué hace?	Ejecuta un programa al que tu código se conecta desde fuera	Aloja tu propio entorno de programación
¿Quién se conecta?	Tu aplicación, por red y puerto	Tu editor, directamente desde dentro
Ejemplo de imagen	postgres:18-alpine	mcr.microsoft.com/devcontainers/base:debian-12

Ideas clave

- Un Dev Container usa Docker para alojar tu propio entorno de programación, no un servicio.
- Tu editor se conecta directamente dentro, guiado por un fichero `.devcontainer/devcontainer.json`.
- La ventana del editor no cambia de aspecto: lo que cambia es dónde corren de verdad la terminal, el depurador y el autocompletado.
- El `devcontainer.json` puede construir el entorno a partir de una image suelta, o de un servicio dentro de un `docker-compose.yml` ya existente.